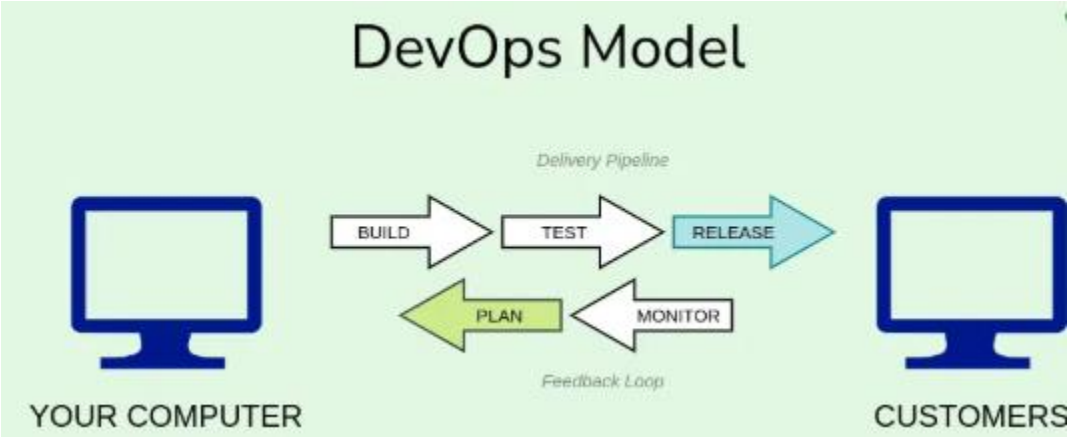
Introduction to DevOps:

The DevOps is a combination of two words, one is software Development, and second is Operations. This allows a single team to handle the entire application lifecycle, from development to **testing, deployment**, and **operations**. DevOps helps you to reduce the disconnection between software developers, quality assurance (QA) engineers, and system administrators.

DevOps Model



Delivery Pipeline

The pipeline represents the different stages that software goes through before it is released to production. These stages might typically include:

- Build: The stage where the software code is compiled and packaged into a deployable unit.
- Test: The stage where the software is rigorously tested to ensure it functions as expected and identifies any bugs.
- Release: The stage where the software is deployed to production for end users.

Feedback Loop

The loop indicates that information and learning's from the production environment are fed back into the earlier stages of the pipeline. This feedback can be used to improve the software development process and future releases.

Key Principles of DevOps

1. Collaboration: DevOps encourages closer collaboration between development and operations teams to enhance communication and ensure smoother processes.

2. Automation: Automation is a cornerstone of DevOps. It involves automating manual tasks like testing, deployment, and infrastructure management to increase efficiency and reduce human error.

3. Continuous Integration and Continuous Deployment (CI/CD): CI/CD practices allow code changes to be integrated and deployed automatically. With CI, code changes are frequently integrated into the main branch, tested, and deployed. CD ensures that software can be deployed to production at any time.

4. Monitoring and Feedback: Continuous monitoring of the system in real-time helps identify issues early and optimize performance. Feedback loops allow developers and operations teams to continually improve the software and processes.

DevOps Tools

1. Jenkins
- Purpose: Continuous Integration and Continuous Delivery (CI/CD)
- Features: Extensive plugin ecosystem, automation of build and deployment processes, and scalability.

2. Docker
- Purpose: Containerization
- Features: Simplifies application deployment, ensures consistency across environments, and supports microservices architecture.

3. Kubernetes
- Purpose: Container Orchestration
- Features: Automates deployment, scaling, and management of containerized applications.

4. Terraform
- Purpose: Infrastructure as Code (IaC)
- Features: Manages infrastructure across multiple cloud providers, ensures reproducibility, and supports version control.

5. Ansible
- Purpose: Configuration Management and Automation
- Features: Simple syntax (YAML), agentless architecture, and scalability.

6. Prometheus
- Purpose: Monitoring and Alerting
- Features: Collects and stores metrics, powerful querying language (PromQL), and integration with Grafana for visualization.

7. GitLab
- Purpose: Source Code Management and CI/CD
- Features: Integrated DevOps platform, supports code versioning, CI/CD pipelines, and project management tools.

8. ELK Stack (Elasticsearch, Logstash, Kibana)
- Purpose: Logging and Analytics
- Features: Real-time data search and analysis, log aggregation, and powerful visualization capabilities.

9. Azure DevOps

- Purpose: End-to-End DevOps Solution
- Features: Supports CI/CD, project management, and integrates with various development and monitoring tools.

10. Nagios
- Purpose: Infrastructure Monitoring
- Features: Monitors network services, server resources, and provides alerts for potential issues.

DevOps Lifecycle

DevOps is about more than just automating builds or deployments. The DevOps lifecycle covers several stages:

1. **Plan**: The process starts with planning, where requirements are gathered, and project goals are outlined.
2. **Develop**: Developers write code and use version control systems to manage it.
3. **Build**: Build tools like Maven or Gradle compile the code, run tests, and create the application package.
4. **Test**: Automated testing tools ensure the code works as expected and meets quality standards.
5. **Release**: The code is deployed to staging or production environments, often with automated deployment tools like Jenkins or Azure DevOps.
6. **Deploy**: The application is deployed to production.
7. **Operate**: Once deployed, the application is continuously monitored for issues or performance improvements.
8. **Monitor**: Monitoring tools track the application's health, helping teams to catch issues early and continuously improve the product.

Benefits of DevOps

- **Faster Time to Market**: By automating manual tasks and integrating development and operations, DevOps speeds up the entire software delivery process.
- **Improved Collaboration**: DevOps fosters a culture of communication and collaboration between teams, improving efficiency and breaking down silos.
- **Higher Quality**: Continuous testing, integration, and monitoring help improve the quality and reliability of applications.
- **Scalability and Flexibility**: With automated deployment pipelines and cloud-based infrastructure, DevOps allows organizations to scale their applications easily and adapt quickly to changes.

Experiment No: 1. Introduction to Maven and Gradle: Overview of Build Automation Tools, Key Differences Between Maven and Gradle, Installation and Setup.

Aim: The aim of this experiment is to introduce Maven and Gradle as build automation tools, highlighting their key differences, and providing a hands-on guide to their installation and setup.

Procedure:

Step 1: Introduction to Build Automation Tools

These tools automate repetitive tasks like compiling, testing, packaging, and deploying software. Maven and Gradle are two popular tools used for this purpose in Java-based projects.

- **Maven**: An XML-based tool that follows a convention-over-configuration principle.
- **Gradle**: A more flexible build tool that uses Groovy or Kotlin for configuration and is designed for higher performance.

Step 2: Key Differences Between Maven and Gradle

- **Configuration**:
  - Maven uses XML for configuration (pom.xml).
  - Gradle uses Groovy or Kotlin-based DSL for configuration (build.gradle or build.gradle.kts).
- **Performance**:
  - Gradle tends to be faster due to incremental builds and parallel execution.
  - Maven follows a more rigid build process that can be slower.
- **Flexibility**:
  - Gradle is highly flexible, allowing custom plugins and task configurations.
  - Maven offers a structured and standardized approach but is less flexible.
- **Dependency Management**:
  - Both tools manage dependencies but Maven uses repositories and Gradle leverages a more powerful and flexible dependency resolution system.

Step 3: Installing Maven and Gradle on Ubuntu 24.04

Install Maven

1. **Update package index**:

   sudo apt update

2. **Install Maven**:

   sudo apt install maven

3. **Verify Maven installation**:

mvn –v

Install Gradle

1. **Install required dependencies**:

   sudo apt install openjdk-11-jdk wget

2. **Download Gradle**:

   wget https://services.gradle.org/distributions/gradle-7.5.1-bin.zip

3. **Unzip the downloaded Gradle archive**:

   sudo unzip gradle-7.5.1-bin.zip -d /opt

4. **Set up environment variables**:

   - Open the `.bashrc` file for editing:

     nano ~/.bashrc

   - Add the following lines to set the `GRADLE_HOME` environment variable and update the `PATH`:

     export GRADLE_HOME=/opt/gradle-7.5.1
     export PATH=$PATH:$GRADLE_HOME/bin

   - Save and close the file, then reload `.bashrc`:

     source ~/.bashrc

5. **Verify Gradle installation**:

   gradle -v

Outputs:

Install Maven:

Output: The system installs Maven successfully.

Verify Maven installation:

Output:

Apache Maven 3.8.6 (e.g., version number may vary)

Maven home: /usr/share/maven

Java version: 11.0.9.1, vendor: AdoptOpenJDK, runtime: /usr/lib/jvm/adoptopenjdk-11-hotspot amd64

Default locale: en_US, platform encoding: UTF-8

OS name: "linux", version: "5.4.0-90-generic", arch: "amd64", family: "unix"

Install Gradle:

Output: Gradle archive is downloaded and extracted.

Verify Gradle installation:

Output:

Gradle 7.5.1

Build time: 2022-02-21 11:55:39 UTC

Revision: a1b8b0bcb28fe42893b9bbd8f1b1a2724a98acb0

Kotlin: 1.5.30

Groovy: 3.0.9

Ant: Apache Ant(TM) version 1.10.11 compiled on September 27 2021

JVM: 11.0.9.1 (AdoptOpenJDK 11.0.9.1+1)

OS: Linux 5.4.0-90-generic amd64

Experiment No: 2. Working with Maven: Creating a Maven Project, Understanding the POM File, Dependency Management and Plugins**.**

Aim: The aim of this experiment is to understand Maven by creating a Maven project, exploring the POM file, managing dependencies, and using plugins for efficient project build and automation.

Procedure:

Step 1: Install Maven

1. **Update package index**:
   sudo apt update
2. **Install Maven**:
   sudo apt install maven
3. **Verify Maven installation**:
   mvn –v

Step 2: Create a Maven Project

1. **Open a terminal and run:**
   mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenProject -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
2. **Navigate into the project directory:**
   cd MyMavenProject

Step 3: Understand the pom.xml File

1. **Open pom.xml using a text editor like nano** or **VS Code:**
   nano pom.xml
2. **Important sections:**
   a. <groupId>, <artifactId>, <version> → Define project identity.
   b. <dependencies> → Manage external libraries.
   c. <build> and <plugins> → Configure the build process.

Step 4: Add Dependencies

1. **Open pom.xml:**
   nano pom.xml
2. **Add JUnit dependency:**

```
<dependencies>
  <dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>4.13.2</version>
    <scope>test</scope>
```

```
    </dependency>
  </dependencies>
```

3. **Save (CTRL+X → Y → ENTER).**
4. **Update dependencies:**
   mvn clean install

Step 5: Compile the Project

mvn compile

Step 6: Run the Project

mvn test

Step 7: Use Maven Plugins

Open pom.xml and add maven-compiler-plugin:

```
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.8.1</version>
      <configuration>
        <source>1.8</source>
        <target>1.8</target>
      </configuration>
    </plugin>
  </plugins>
</build>
```

Recompile:

mvn clean compile

Step 8: Package the Project

mvn package

Step 9: Run the Packaged JAR (If Executable)

java -jar target/MyMavenProject-1.0-SNAPSHOT.jar

Outputs:

Install Maven:

Output: The system installs Maven successfully.

Verify Maven installation:

Output:

Apache Maven 3.8.6 (e.g., version number may vary)

Maven home: /usr/share/maven

Java version: 11.0.9.1, vendor: AdoptOpenJDK, runtime: /usr/lib/jvm/adoptopenjdk-11-hotspot amd64

Default locale: en_US, platform encoding: UTF-8

OS name: "linux", version: "5.4.0-90-generic", arch: "amd64", family: "unix"

Create a Maven Project:

Output:

[INFO] Scanning for projects...

[INFO] Creating project directory /home/user/MyMavenProject

[INFO] Generating project in Batch mode

[INFO] ----------< com.example:MyMavenProject >----------

[INFO] Building MyMavenProject 1.0-SNAPSHOT

[INFO] ---------------------------[jar]---------------------------

[INFO] BUILD SUCCESS

[INFO] Total time: 2.456 s

[INFO] Finished at: 2025-02-05T10:30:00Z

nano pom.xml:

Output(Partial pom.xml Content):

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
          http://maven.apache.org/xsd/maven-4.0.0.xsd">
```

```
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.example</groupId>
    <artifactId>MyMavenProject</artifactId>
    <version>1.0-SNAPSHOT</version>
    <dependencies>
        <dependency>
            <groupId>junit</groupId>
            <artifactId>junit</artifactId>
            <version>4.13.2</version>
            <scope>test</scope>
        </dependency>
    </dependencies>
</project>
```

Compile the Project:

Output:

```
[INFO] Scanning for projects...
[INFO] --------------------< com.example:MyMavenProject >--------------------
[INFO] Building MyMavenProject 1.0-SNAPSHOT
[INFO] Compiling 1 source file to /home/user/MyMavenProject/target/classes
[INFO] BUILD SUCCESS
[INFO] Total time: 1.567 s
```

Run Unit Tests:

Output:

```
[INFO] Scanning for projects...
[INFO] --------------------< com.example:MyMavenProject >--------------------
[INFO] Running com.example.AppTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.254 s
[INFO] BUILD SUCCESS
```

Build & Package the Project:

Output:

```
[INFO] Scanning for projects...
[INFO] Building MyMavenProject 1.0-SNAPSHOT
```

[INFO] Compiling 1 source file to /home/user/MyMavenProject/target/classes
[INFO] Packaging JAR
[INFO] Successfully built JAR file: /home/user/MyMavenProject/target/MyMavenProject-1.0-SNAPSHOT.jar
[INFO] BUILD SUCCESS

Run the Packaged JAR (If Executable):
Output:
Hello World!

Experiment No: 3 Working with Gradle: Setting Up a Gradle Project, Understanding Build Scripts (Groovy and Kotlin DSL), Dependency Management and Task Automation**.**

Aim: To provide a comprehensive understanding of Gradle, including project setup, build script configuration (Groovy and Kotlin DSL), dependency management, and task automation for efficient software development.

Procedure:

Step 1: Install Java (JDK)

```
sudo apt update
sudo apt install openjdk-17-jdk -y
```

Verify installation:

```
java -version
```

Step 2: Install Gradle

Install required dependencies:

sudo apt install openjdk-11-jdk wget

Download Gradle:

wget https://services.gradle.org/distributions/gradle-7.5.1-bin.zip

Unzip the downloaded Gradle archive:

sudo unzip gradle-7.5.1-bin.zip -d /opt

Set up environment variables:

- Open the `.bashrc` file for editing:

  nano ~/.bashrc

- Add the following lines to set the `GRADLE_HOME` environment variable and update the `PATH`:

  export GRADLE_HOME=/opt/gradle-7.5.1
  export PATH=$PATH:$GRADLE_HOME/bin

- Save and close the file, then reload `.bashrc`:

  source ~/.bashrc

Verify Gradle installation:

gradle -v

Step 3: Create a New Gradle Project

1. **Navigate to your workspace:**
   mkdir ~/gradle-projects && cd ~/gradle-projects
2. **Generate a new Gradle project using the init command:**
   gradle init
3. **Select the project type:**
   - Type: application
   - Language: Java
   - Build script DSL: Groovy or Kotlin

Step 4: Understanding Build Scripts

- The generated project will have a build.gradle (Groovy) or build.gradle.kts (Kotlin) file.
- Example Groovy DSL (build.gradle):

```
plugins {
    id 'application'
}

application {
    mainClass = 'com.example.App'
}

dependencies {
    implementation 'org.apache.commons:commons-lang3:3.12.0'
```

```
}
```

- Example Kotlin DSL (build.gradle.kts):

```
plugins {
    application
}

application {
    mainClass.set("com.example.App")
}

dependencies {
    implementation("org.apache.commons:commons-lang3:3.12.0")
}
```

Step 5: Managing Dependencies

- Add dependencies in dependencies block.
- Example (adding JUnit for testing):

```
dependencies {
    testImplementation 'junit:junit:4.13.2'
}
```

- **Run gradle dependencies to check all dependencies:**

gradle dependencies

Step 6: Automating Tasks

- **List available tasks:**

gradle tasks

- **Compile the project:**

gradle build

- **Run the application:**

gradle run

- **Clean the project:**

gradle clean

Step 7: Writing Custom Tasks

Define a custom Gradle task in build.gradle:

```
task hello {
    doLast {
```

```
        println 'Hello, Gradle!'
    }
}
```

Run the custom task:

gradle hello

Step 8: Running Tests

1. Write a test file inside src/test/java/ (for Java projects).
2. Run tests:
   gradle test

Create a Test File:

Navigate to the src/test/java/ directory:

mkdir -p src/test/java/com/example

Create a test file, e.g., AppTest.java:

nano src/test/java/com/example/AppTest.java

Add the following test code:

```
import org.junit.Test;
import static org.junit.Assert.*;
public class AppTest {
    @Test
    public void testAddition() {
        int sum = 2 + 3;
        assertEquals(5, sum);
    }
    @Test
    public void testMultiplication() {
        int product = 2 * 3;
        assertEquals(6, product);
    }
}
```

Save the file (CTRL + X, then Y, then Enter).

Outputs:

Install Java (JDK)

Output:

```
openjdk version "17.0.x" 2024-XX-XX
OpenJDK Runtime Environment ...
OpenJDK 64-Bit Server VM ...
```

Install Gradle

Output:

```
Welcome to Gradle 7.5.1!
------------------------------------------------------------
Gradle 7.5.1
Build time:   2024-XX-XX XX:XX:XX UTC
Kotlin:       1.6.21
Groovy:       3.0.9
JVM:          17.0.x (Ubuntu)
OS:           Linux 5.x.x-xx-generic amd64
```

Create a New Gradle Project

Output:

```
Starting a Gradle Daemon, X busy Daemons could not be reused, use --status for details
Select type of project to generate:
1: basic
2: application
3: library
4: Gradle plugin
Enter selection (default: basic) [1..4]:
```

Understanding Build Scripts

Output:

```
plugins {
    id 'application'
}

application {
    mainClass = 'com.example.App'
}
dependencies {
    implementation 'org.apache.commons:commons-lang3:3.12.0'
}
```

Managing Dependencies

Output:

```
------------------------------------------------------------
Root project
------------------------------------------------------------
```

+--- org.apache.commons:commons-lang3:3.12.0
+--- junit:junit:4.13.2

Automating Tasks
Output:
Hello, Gradle!

Writing Custom Tasks
Output:
Hello, Gradle!

Running Tests
Output:
com.example.AppTest > testAddition PASSED
com.example.AppTest > testMultiplication PASSED
BUILD SUCCESSFUL in 3s
2 actionable tasks: 2 executed

Experiment No: 4. Practical Exercise: Build and Run a Java Application with Maven, Migrate the Same Application to Gradle.

Aim: The aim is to build and run a Java application with **Maven**, then migrate it to **Gradle** to compare and understand the differences in build automation and dependency management.

Procedure:
Step 1: Install Java (JDK)
sudo apt update
sudo apt install openjdk-17-jdk –y
Install Maven
sudo apt install maven –y

Step 2: Create a Maven Project
mvn archetype:generate -DgroupId=com.example -DartifactId=myapp
-DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
Step 3: Navigate to the Project Directory
cd myapp

Step 4: Build the Maven Project
mvn package

Step 5: Run the Java Application
java -cp target/myapp-1.0-SNAPSHOT.jar com.example.App

Step 6: Install Gradle
wget https://services.gradle.org/distributions/gradle-8.0.2-bin.zip
sudo unzip gradle-8.0.2-bin.zip -d /opt
echo 'export GRADLE_HOME=/opt/gradle-8.0.2' >> ~/.bashrc
echo 'export PATH=$GRADLE_HOME/bin:$PATH' >> ~/.bashrc
source ~/.bashrc

Step 7: Convert Maven Project to Gradle
gradle init

Step 8: Verify build.gradle File
cat build.gradle

Step 9: Build the Gradle Project
gradle build

Step 10: Run the Application Using Gradle
gradle run

Outputs:
Install Java (JDK)
Output:
openjdk version "17.0.x"
OpenJDK Runtime Environment ...
OpenJDK 64-Bit Server VM ...
Install Maven
Output:
Apache Maven 3.x.x
Maven home: /usr/share/maven
Java version: 17.0.x, vendor: Ubuntu

Create a Maven Project
Output:
[INFO] BUILD SUCCESS

Build the Maven Project
Output:

[INFO] BUILD SUCCESS

Run the Java Application
Output:
Hello, World!

Migrating the Application from Maven to Gradle

Install Gradle
Output:
Welcome to Gradle 8.0.2!

Convert Maven Project to Gradle
Output:
Select type of project to generate:
1: basic
2: application
3: library
...

Verify build.gradle File
Output:

```
plugins {
   id 'application'
}

application {
   mainClass = 'com.example.App'
}

dependencies {
   implementation 'org.apache.commons:commons-lang3:3.12.0'
}
```

Build the Gradle Project
Output:
BUILD SUCCESSFUL

Run the Application Using Gradle
Output:

Hello, World!

Experiment No: 5. Introduction to Jenkins: What is Jenkins?, Installing Jenkins on Local or Cloud Environment, Configuring Jenkins for First Use.

Aim: The aim is to introduce Jenkins, explain its purpose, guide through installing Jenkins on a local or cloud environment, and demonstrate how to configure Jenkins for first-time use to automate build and deployment processes.

Procedure:

What is Jenkins?

Jenkins is an open-source automation server primarily used for continuous integration (CI) and continuous delivery (CD). It allows developers to automatically build, test, and deploy their software by integrating with various version control tools, such as Git, and other tools used for testing and deployment. It helps automate repetitive tasks in software development, making the process faster and more efficient.

Key Features:

- **CI/CD Pipelines:** Automates the process of building, testing, and deploying software.
- **Extensibility:** A vast ecosystem of plugins to support different build, testing, and deployment tools.
- **Distributed Builds:** Can distribute builds across multiple nodes to speed up processing.

Installing Jenkins on Local Environment

Step 1.Update the system:

sudo apt update
sudo apt upgrade –y

Step 2. Install Open JDK 17
sudo apt install openjdk-17-jdk –y

Check java version
java -version

Set JAVA_HOME (Optional):
echo "export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64" >> ~/.bashrc
source ~/.bashrc

Verify by running:
echo $JAVA_HOME

Step 3. Add Jenkins Repository:
wget -q -O - https://pkg.jenkins.io/keys/jenkins.io.key | sudo tee /etc/apt/trusted.gpg.d/jenkins.asc
echo "deb http://pkg.jenkins.io/debian/ stable main" | sudo tee -a /etc/apt/sources.list.d/jenkins.list

Update System and Install Jenkins:
sudo apt update
sudo apt install jenkins –y

Start Jenkins:
sudo systemctl start Jenkins

Enable Jenkins on Boot:
sudo systemctl enable Jenkins

Check Jenkins Status:
sudo systemctl status Jenkins

Install Jenkins on Cloud (e.g., AWS EC2)

1. **Launch EC2 Instance**:
   - In AWS, launch an EC2 instance running Ubuntu 24.04 (or any supported Ubuntu version).
   - Make sure the security group allows inbound traffic on ports **22** (SSH) and **8080** (Jenkins HTTP).
2. **Connect to the EC2 Instance**: SSH into your instance:
   ssh -i "your-key.pem" ubuntu@your-ec2-public-ip
3. **Follow the same steps as the local installation** to install **JDK 17** and **Jenkins**.
4. **Access Jenkins Web Interface**:
   - Open the browser and go to http://your-ec2-public-ip:8080.
   - Retrieve the Jenkins unlock key:

     sudo cat /var/lib/jenkins/secrets/initialAdminPassword

   - Paste the unlock key into the web interface to proceed.

Step 4. Configuring Jenkins for First Use with JDK 17

1. **Unlock Jenkins**:
   - After accessing Jenkins in the browser, enter the unlock key obtained from the command above.

2. **Install Suggested Plugins**:
    - Choose **Install suggested plugins** when prompted, which installs the most common plugins.
3. **Create an Admin User**:
    - You will be asked to create an admin user. You can also skip this step and use the default admin credentials if you prefer.
4. **Jenkins Dashboard**:
    - After the setup, you'll be taken to the Jenkins Dashboard, where you can start creating jobs.

Step 5. Verify Jenkins is Using JDK 17

To ensure Jenkins is running with JDK 17:

1. **Check System Information**:
    - Go to **Manage Jenkins** > **System Information**.
    - Under the **Java** section, verify that the **Java version** is JDK 17.
2. **Configure JDK 17 for Jenkins (if needed)**:
    - If Jenkins is not using JDK 17 by default, you can configure the JDK path under **Manage Jenkins** > **Global Tool Configuration**.
    - Under **JDK** section, specify the path to the JDK 17 installation (e.g., /usr/lib/jvm/java-17-openjdk-amd64).

Step 6. Test Jenkins Setup (Create a Simple Job)

1. **Create a New Job**:
    - From the Jenkins dashboard, click on **New Item**.
    - Enter a name for the job (e.g., "Test Job"), select **Freestyle project**, and click **OK**.
2. **Configure the Job**:
    - Add a **Build step** (e.g., **Execute Shell**).
    - Add a simple command, like:

      echo "Hello Jenkins!"

3. **Save and Build**:
    - Click **Save** and then **Build Now**.
    - The build will run, and you can view the console output by clicking on the build number in the **Build History**.

Outputs:

Install JDK 17
Output:
openjdk version "17.0.1" 2021-10-19
OpenJDK Runtime Environment (build 17.0.1+12-39)

OpenJDK 64-Bit Server VM (build 17.0.1+12-39, mixed mode)

Set JAVA_HOME (Optional)
Output:
/usr/lib/jvm/java-17-openjdk-amd64

Install Jenkins
Output:
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following additional packages will be installed:
...
Need to get 85.7 MB of archives.
After this operation, 317 MB of additional disk space will be used.

Start Jenkins
Output:
jenkins.service - Jenkins Continuous Integration Server
Loaded: loaded (/etc/systemd/system/jenkins.service; enabled; vendor preset: enabled)
Active: active (running) since Mon 2025-02-06 12:00:00 UTC; 10s ago

Enable Jenkins to start on boot
Output:
Created symlink /etc/systemd/system/multi-user.target.wants/jenkins.service
/etc/systemd/system/jenkins.service.

Access Jenkins Web Interface
Output:
For Example: 8a5d6c3f6d2e47cb9efb6b024a09b5f9

Install Suggested Plugins
Output:
Downloading plugin: git
Downloading plugin: workflow-aggregator
….

Create Admin User
Output:
"Jenkins is ready!"
Verify Jenkins Setup and Test Job
Output:
Hello Jenkins!

Experiment No: 6. Continuous Integration with Jenkins: Setting Up a CI Pipeline, Integrating Jenkins with Maven/Gradle, Running Automated Builds and Tests.

Aim: The aim of this experiment is to set up a local Jenkins CI pipeline that automatically builds, tests, and packages Java applications using Maven or Gradle—demonstrating how continuous integration can improve code quality and streamline development workflows.

Procedure:

Step 1: Install Java (JDK 11)

```
sudo apt update
sudo apt install openjdk-11-jdk –y
```

Step 2: Install Jenkins on Ubuntu

1. Add Jenkins Repository & Install Jenkins

```
curl -fsSL https://pkg.jenkins.io/debian/jenkins.io-2023.key | sudo tee \
/usr/share/keyrings/jenkins-keyring.asc > /dev/null

echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \
https://pkg.jenkins.io/debian binary/" | sudo tee \
/etc/apt/sources.list.d/jenkins.list > /dev/null

sudo apt update
sudo apt install jenkins –y
```

2. Start and Enable Jenkins

```
sudo systemctl enable --now Jenkins
```

3. Check Jenkins Status

```
sudo systemctl status Jenkins
```

Step 3: Unlock and Access Jenkins

1. Get the Initial Admin Password

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

2. Access Jenkins UI

http://localhost:8080

3. Enter the Password and Install Suggested Plugins.

Step 4: Install Required Plugins

- Go to **Manage Jenkins → Manage Plugins**.
- Install:
  - **Maven Integration Plugin**

- **Gradle Plugin**
- **Git Plugin** (for version control)

Step 5: Configure Java, Maven, and Gradle in Jenkins

1. Go to **Manage Jenkins → Global Tool Configuration**.
2. Configure:
   - **JDK 11**: Ensure Java 11 is selected (/usr/lib/jvm/java-11-openjdk-amd64).
   - **Maven**: Install Maven if needed:
     sudo apt install maven -y
   - **Gradle**: Install Gradle:
     sudo apt install gradle -y
   - Verify installations:
     mvn -version
     gradle –version

Step 6: Create a Jenkins Job (Pipeline)

1. Click **New Item → Freestyle Project / Pipeline**.
2. Name it and click **OK**.

Step 7: Connect Jenkins with GitHub/GitLab

1. Under **Source Code Management**, select **Git**.
2. Enter your repository URL (e.g., https://github.com/your-repo.git).
3. Add SSH keys or credentials if needed.
4. Choose the branch (e.g., main).

Step 8: Configure Maven/Gradle Build

1. For Maven Projects:
   - Under Build, click Add build step → Invoke top-level Maven targets.
   - Set Goals to:
     clean install
2. For Gradle Projects:
   - Under Build, click Add build step → Invoke Gradle script.
   - Set Tasks to:
     clean build

Step 9: Run Automated Tests

- Jenkins will automatically run tests using:
  - **JUnit, TestNG (Maven projects)**
  - **JUnit (Gradle projects)**
- Configure test reports under **Post-build Actions → Publish JUnit Test Result Report**.

Step 10: Automate Builds Using Triggers

In **Build Triggers**, enable:

- **Poll SCM** (H/5 * * * *) → Checks for changes every 5 minutes.
- **GitHub webhook** (if using GitHub).
- **Build Periodically** (if needed).

Step 11: Run and Monitor Builds

- Click **Build Now** to trigger a manual build.
- Check **Console Output** for errors.
- View **Build History** and **Test Results**.

Outputs:

Install Java

Output:

openjdk version "11.0.x" 2024-XX-XX
OpenJDK Runtime Environment (build 11.0.x+XX)
OpenJDK 64-Bit Server VM (build 11.0.x+XX, mixed mode)

Install Jenkins on Ubuntu

Output:

jenkins.service - Jenkins Continuous Integration Server
Loaded: loaded (/lib/systemd/system/jenkins.service; enabled; vendor preset: enabled)
Active: active (running) since Mon 2024-XX-XX XX:XX:XX UTC

Unlock and access Jenkins

Output:

Expected UI: Jenkins setup page requesting initial admin password.

Install Required Plugins

Output:

Expected UI Output:

✔ Installed Plugins:

- Maven Integration Plugin
- Gradle Plugin
- Git Plugin

Configure Java, Maven, and Gradle in Jenkins

Output:

openjdk version "11.0.x"
Setting up maven (3.x.x) ...
Apache Maven 3.x.x (Red Hat 4.x)
Maven home: /usr/share/maven

Java version: 11.x.x
Gradle 7.x.x
Java version: 11.x.x

Create a Jenkins Job (Pipeline)
Output:
Expected UI Output :

✔ New project created successfully.

Connect Jenkins with GitHub/GitLab
Output:
Expected UI Output:

✔ Repository Connected Successfully.

Configure Maven/Gradle Build
Output:
Maven:
[INFO] BUILD SUCCESS
Total time: 10.345 s
Gradle:
BUILD SUCCESSFUL in 7s

Run Automated Tests
Output:

T E S T S

Running com.example.tests.MyAppTest
Tests run: 10, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS

Automate Builds Using Triggers
Output:
✔ Expected UI Output:

- Poll SCM: (H/5 * * * *) configured.
- GitHub Webhook enabled

Run and Monitor Builds
Output:
Expected Console Output:

Started by user Admin
Building in workspace /var/lib/jenkins/workspace/MyProject
Checking out from SCM...
Running Maven Build...
Tests Passed.
[INFO] BUILD SUCCESS

Expected UI Output:
✅ Build #1 – Success

Experiment No: 7 Configuration Management with Ansible: Basics of Ansible: Inventory, Playbooks, and Modules, Automating Server Configurations with Playbooks, Hands-On: Writing and Running a Basic Playbook.

Aim: To set up and automate server configurations using **Ansible** on a local machine by understanding **inventory, playbooks, and modules**, and executing a playbook to install and manage services like **Nginx**.

Procedure:
Step 1: Install Ansible
sudo apt install ansible –y

Step 2: Configure Inventory File
Create an inventory file (inventory.ini)
nano inventory.ini

Add the following content:
[local]
localhost ansible_connection=local
(Save and exit: **CTRL + X → Y → ENTER**)

Test Ansible Connectivity
ansible all -m ping -i inventory.ini

Step 3: Understand Ansible Modules
Modules are reusable scripts for automation.
Example: Using the apt Module to Install a Package
ansible localhost -m apt -a "name=nginx state=present" –become
Example: Using the command Module
ansible localhost -m command -a "uptime"

Step 4: Create a Basic Playbook
A **playbook** is a YAML file that defines tasks.
Create a Playbook (setup.yml)
nano setup.yml

Add the following content:

```
- name: Configure Local Machine
  hosts: local
  become: yes  # Run tasks as sudo
  tasks:
    - name: Update package lists
      apt:
        update_cache: yes

    - name: Install Nginx
      apt:
        name: nginx
        state: present

    - name: Start Nginx service
      service:
        name: nginx
        state: started
```

(Save and exit: **CTRL + X → Y → ENTER**)

Step 5: Run the Playbook
ansible-playbook -i inventory.ini setup.yml

Step 6: Verify the Playbook Execution
Check Nginx Service
systemctl status nginx
Open Web Browser and Visit:
http://localhost

Outputs:
Install Ansible
Output:

```
ansible [core 2.x.x]
config file = /etc/ansible/ansible.cfg
python version = 3.x.x
```

Configure Inventory File
Output:

```
localhost | SUCCESS => {
   "changed": false,
   "ping": "pong"
}
```

Understand Ansible Modules
Output:

```
Using the apt Module
localhost | SUCCESS => {
   "changed": true,
   "msg": "Nginx installed"
}
```

```
Using the command Module
localhost | SUCCESS => {
   "changed": true,
   "stdout": "14:32:14 up  2:05,  1 user,  load average: 0.00, 0.01, 0.05"
}
```

Create a Basic Playbook
Output:

```
Playbook is created
```

Run the Playbook
Output:

```
PLAY [Configure Local Machine] ***************************************************

TASK [Update package lists] ******************************************************
ok: [localhost]

TASK [Install Nginx] *************************************************************
changed: [localhost]

TASK [Start Nginx service] *******************************************************
changed: [localhost]

PLAY RECAP
*********************************************************************
localhost            : ok=3    changed=2    unreachable=0    failed=0
```

Verify the Playbook Execution
Output:

```
Nginx Service
```

- nginx.service - A high performance web server
  Loaded: loaded (/lib/systemd/system/nginx.service; enabled)
  Active: active (running)

Web Browser

- You should see the default **Nginx Welcome Page**.

Experiment No: 8 Practical Exercise: Set Up a Jenkins CI Pipeline for a Maven Project, Use Ansible to Deploy Artifacts Generated by Jenkins.

Aim: To **set up a Jenkins CI pipeline** for a Maven project, automating the **build and test process**, and then use **Ansible** to deploy the generated artifacts to a target server, ensuring a streamlined CI/CD workflow.

Procedure:

Step 1: Install Java

sudo apt update && sudo apt upgrade -y

sudo apt install openjdk-11-jdk –y

Step 2: Install and Configure Jenkins

Install Jenkins:

wget -O- https://pkg.jenkins.io/debian/jenkins.io.key | sudo tee /usr/share/keyrings/jenkins-keyring.asc

echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] https://pkg.jenkins.io/debian binary/" | sudo tee /etc/apt/sources.list.d/jenkins.list > /dev/null

sudo apt update

sudo apt install jenkins –y

Start and Enable Jenkins:

sudo systemctl start jenkins

sudo systemctl enable Jenkins

Get Jenkins Admin Password:

sudo cat /var/lib/jenkins/secrets/initialAdminPassword

Access Jenkins UI:

http://localhost:8080

Step 3: Install and Configure Maven in Jenkins

Install Maven:

sudo apt install maven –y

Add Maven in Jenkins

- Open **Jenkins Dashboard**
- Go to **Manage Jenkins → Global Tool Configuration**
- Under **Maven**, add a new installation
- Name it **Maven-3** and set path to

/usr/share/maven

Step 4: Create a Jenkins CI Pipeline for a Maven Project

Create a Maven Project

- In **Jenkins Dashboard → New Item → Maven Project**
- Name it **Maven-CI**
- Under **Source Code Management**, add a **Git repository**

https://github.com/sample-user/sample-maven-project.git

Under **Build**, enter:
clean package

Run the Jenkins Build

- Click **Build Now**.
- Jenkins fetches the project, compiles the code, runs tests, and generates a **JAR/WAR file** in target/ directory.
-

Step 5: Install and Configure Ansible
Install Ansible:
sudo apt install ansible –y

Configure Ansible Inventory
nano inventory.ini

Add:
[local]
localhost ansible_connection=local
(Save and exit: **CTRL + X → Y → ENTER**)

Step 6: Use Ansible to Deploy the Artifact
Create an Ansible Playbook (deploy.yml)
nano deploy.yml

Add:
- name: Deploy Maven Artifact

```
 hosts: local
 become: yes
 tasks:
   - name: Copy Artifact from Jenkins Workspace
     copy:
       src: /var/lib/jenkins/workspace/Maven-CI/target/myapp.jar
       dest: /opt/myapp.jar
       mode: '0644'

   - name: Run the Java Application
     command: nohup java -jar /opt/myapp.jar &
```

(Save and exit: **CTRL + X → Y → ENTER**)

Step 7: Run the Ansible Playbook to Deploy
ansible-playbook -i inventory.ini deploy.yml

Step 8: Verify Deployment
Check Running Java Process
ps aux | grep java

Access the Application (If Web-Based)
curl http://localhost:8080

Outputs:
Install Java
Output:
openjdk version "11.0.x"
OpenJDK Runtime Environment (build 11.0.x+XX)
OpenJDK 64-Bit Server VM (build 11.0.x+XX, mixed mode)

Install and Configure Jenkins
Output:
Jenkins is now set up

Install and Configure Maven in Jenkins
Output:
Apache Maven 3.x.x

Create a Jenkins CI Pipeline for a Maven Project
Output:
[INFO] Building myapp 1.0

[INFO] Packaging JAR
[INFO] BUILD SUCCESS

Run the Ansible Playbook to Deploy

Output:

TASK [Copy Artifact from Jenkins Workspace] ***********************************
changed: [localhost]

TASK [Run the Java Application] **
changed: [localhost]

PLAY RECAP
****
localhost : ok=2 changed=2 unreachable=0 failed=0

Verify Deployment

Output:

Hello, welcome to my application!

Experiment No: 9 Introduction to Azure DevOps: Overview of Azure DevOps Services, Setting Up an Azure DevOps Account and Project.

Aim: To understand the **fundamentals of Azure DevOps**, explore its core **services and features**, and learn how to **set up an Azure DevOps account and create a project** for managing software development workflows efficiently.

Procedure:

Step 1: Understanding Azure DevOps Services

Azure DevOps is a cloud-based DevOps platform that helps teams plan, develop, test, and deploy applications. It includes:

1. Azure Repos – Git repositories for version control
2. Azure Pipelines – CI/CD automation
3. Azure Boards – Agile project management
4. Azure Artifacts – Package management
5. Azure Test Plans – Manual and automated testing

Output: Understanding the purpose of each Azure DevOps service.

Step 2: Create an Azure DevOps Account

Go to Azure DevOps Portal

- Open a browser and visit:

https://dev.azure.com

Sign in or Sign Up

- If you have a Microsoft account, **sign in**
- If not, **create a free Microsoft account**

Output: Logged into the Azure DevOps portal.

Step 3: Create a New Azure DevOps Project
Click on "Create a Project"

- Enter Project Name (e.g., MyDevOpsProject)
- Choose Visibility: Private (only team members) or Public

Select Version Control and Work Item Process

- Version Control: Git
- Work Item Process: Agile, Scrum, CMMI, or Basic

Output: A new DevOps project is successfully created.

Step 4: Explore Key Azure DevOps Features
Navigate to "Azure Repos"

- Click on **"Repos"**
- Clone the repository using Git:

git clone https://dev.azure.com/your-org-name/MyDevOpsProject/_git/MyDevOpsProject

Output: Local repository is set up.

Create a Simple Pipeline in "Azure Pipelines"

- Click **"Pipelines" → "New Pipeline"**
- Choose **"GitHub" or "Azure Repos Git"**
- Select a **Starter Pipeline**
- Click **"Save and Run"**

Output: CI/CD pipeline is triggered.

Experiment No: 10 Creating Build Pipelines: Building a Maven/Gradle Project with Azure Pipelines, Integrating Code Repositories (e.g., GitHub, Azure Repos), Running Unit Tests and Generating Reports.

Aim: To set up **Azure Pipelines** for automating the **build, test, and reporting process** of a **Maven/Gradle project**, integrating it with a code repository (GitHub or Azure Repos), and ensuring code quality through **unit testing and report generation**.

Procedure:
Step 1: Set Up the Code Repository
You can use GitHub or Azure Repos to store your Maven/Gradle project.
Clone the Repository Locally

If using GitHub:
git clone https://github.com/your-username/your-repo.git
cd your-repo

If using Azure Repos:
git clone https://dev.azure.com/your-org-name/your-project/_git/your-repo
cd your-repo

Output:

```
Cloning into 'your-repo'...
remote: Enumerating objects: 100%, done.
Receiving objects: 100%, done.
```

Step 2: Create an Azure DevOps Pipeline

- Go to Azure DevOps Portal → Navigate to Pipelines
- Click "New Pipeline"
- Select your repository (GitHub/Azure Repos)
- Choose Maven or Gradle Starter Pipeline

Step 3: Define the Pipeline YAML File
Modify your azure-pipelines.yml file.
For Maven Projects:

```
trigger:
- main  # Runs pipeline on new commits
pool:
  vmImage: 'ubuntu-latest'
steps:
- task: Maven@3
  inputs:
    mavenPomFile: 'pom.xml'
    goals: 'clean package'
    publishJUnitResults: true
    testResultsFiles: '**/surefire-reports/TEST-*.xml'
    javaHomeOption: 'JDKVersion'
    mavenVersionOption: 'Default'
    sonarQubeRunAnalysis: false
```

For Gradle Projects:

```
trigger:
- main
pool:
  vmImage: 'ubuntu-latest'
```

```
steps:
- script: |
    chmod +x gradlew
    ./gradlew build test
  displayName: 'Build and Test with Gradle'
```

Output: Pipeline will now trigger automatically on new commits!

Step 4: Running the Build Pipeline
Commit and Push the Changes

```
git add azure-pipelines.yml
git commit -m "Added Azure Pipeline"
git push origin main
```

Go to Azure DevOps → Pipelines → Click on Your Pipeline → Run it manually
Output in Azure DevOps Console:

```
Job started: Building Maven/Gradle Project...
[INFO] Compiling source files
[INFO] Running unit tests...
BUILD SUCCESSFUL!
```

Step 5: Verify Unit Tests and Reports
Check Test Results
Navigate to Pipelines → Your Build → Tests Tab
Output:
✔ All Tests Passed! (5/5 successful)
Download Reports

- Navigate to Pipelines → Select Build → Artifacts
- Download target/surefire-reports/TEST-*.xml (Maven) or build/reports/tests/test/index.html (Gradle).

Output:
A test report showing passed and failed test cases.

Experiment No: 11 Creating Release Pipelines: Deploying Applications to Azure App Services, Managing Secrets and Configuration with Azure Key Vault, Hands-On: Continuous Deployment with Azure Pipelines.

Aim: To set up an **Azure Release Pipeline** for automating the **deployment of applications** to **Azure App Services**, securely **managing secrets and configurations** using **Azure Key Vault**, and implementing **Continuous Deployment (CD) with Azure Pipelines** for seamless application updates.

Procedure:

Step 1: Prerequisites

- Before starting, ensure you have:
- An Azure DevOps account https://dev.azure.com
- An Azure App Service created in Azure Portal
- An application stored in GitHub/Azure Repos
- Azure CLI installed for managing resources

Step 2: Create a New Release Pipeline

Open Azure DevOps

1. Navigate to Azure DevOps → Pipelines → Releases
2. Click "New Pipeline"

Select a Deployment Template

1. Choose "Azure App Service Deployment"
2. Click "Apply"

Output:

A new release pipeline with an "Azure App Service Deployment" stage.

Step 3: Configure the Release Pipeline

Add an Artifact

1. Click "Add an artifact"
2. Choose "Build Artifacts" from Azure Pipelines or GitHub
3. Select the correct Build Source
4. Click "Add"

Output:

Artifact successfully linked.

Define Deployment Stages

1. Click "Stage 1" → "Deploy Azure App Service"
2. Select Azure Subscription and authenticate
3. Choose App Service Name
4. Set Package or Folder to $(System.DefaultWorkingDirectory)/_your-build-folder/drop
5. Click Save

Output:

Deployment stage configured.

Enable Continuous Deployment

1. Click "Pre-deployment conditions"
2. Enable "Continuous Deployment Trigger"

Output:

Pipeline will trigger automatically on new builds.

Step 4: Manage Secrets with Azure Key Vault
Create an Azure Key Vault
az keyvault create --name myKeyVault --resource-group myResourceGroup --location eastus
Add a Secret
az keyvault secret set --vault-name myKeyVault --name "DatabasePassword" --value "SuperSecure123!"
Output:
Secret successfully stored.
Link Key Vault to Azure Pipeline

1. Go to Pipeline Variables → Add Variable
2. Click "Link secrets from Azure Key Vault"
3. Select your Key Vault
4. Add the required secrets

Output:
Azure Key Vault secrets linked.

Step 5: Trigger and Monitor the Release Pipeline
Start a Release

1. Click "Create a Release"
2. Select "Deploy to App Service"
3. Click "Deploy"

Output:
Deployment started...
Fetching build artifacts...
Deploying to Azure App Service...
Deployment successful!

Verify Deployment

1. Open Azure Portal → App Services → Your App
2. Click "Browse" to open the deployed application

Output:
A running web application in Azure App Services.
Experiment No: 12 Practical Exercise and Wrap-Up: Build and Deploy a Complete DevOps Pipeline, Discussion on Best Practices and Q&A.

Aim: To **build and deploy a complete DevOps pipeline** by integrating **Continuous Integration (CI), Continuous Deployment (CD), and Configuration Management** tools. The exercise includes **hands-on implementation, discussion on best practices, and a Q&A session** to reinforce DevOps principles and streamline software delivery.

Procedure:

Step 1: Set Up Version Control (GitHub/Azure Repos)

Initialize a Git Repository:

If using GitHub:

```
git clone https://github.com/your-username/your-repo.git
cd your-repo
```

If using Azure Repos:

```
git clone https://dev.azure.com/your-org-name/your-project/_git/your-repo
cd your-repo
```

Output:

```
Cloning into 'your-repo'...
remote: Enumerating objects: 100%, done.
Receiving objects: 100%, done.
```

Step 2: Set Up CI/CD Pipeline (Jenkins & Azure Pipelines)

Install Jenkins on Ubuntu

```
sudo apt update && sudo apt install openjdk-11-jdk -y
wget -q -O - https://pkg.jenkins.io/debian/jenkins.io.key | sudo apt-key add -
sudo sh -c 'echo deb http://pkg.jenkins.io/debian-stable binary/ > /etc/apt/sources.list.d/jenkins.list'
sudo apt update && sudo apt install jenkins -y
sudo systemctl start jenkins && sudo systemctl enable jenkins
```

Output:

Jenkins installed and running on http://localhost:8080

Step 3: Configure Build in Jenkins

Create a New Jenkins Pipeline

1. Open Jenkins UI → New Item → Pipeline
2. Configure the Git Repository URL
3. Add a Jenkinsfile in the repo:

```
pipeline {
   agent any
   stages {
      stage('Checkout') {
         steps {
            git 'https://github.com/your-username/your-repo.git'
         }
      }
      stage('Build') {
```

```
        steps {
            sh 'mvn clean package'
        }
    }
    stage('Test') {
        steps {
            sh 'mvn test'
        }
    }
    stage('Deploy') {
        steps {
            sh 'scp target/myapp.jar user@server:/opt/myapp.jar'
        }
    }
  }
}
```

Output:

Pipeline runs successfully, building and testing the application.

Step 4: Deploy to Azure App Services

Create an Azure App Service

az webapp create --name MyAppService --resource-group MyResourceGroup --plan MyAppServicePlan --runtime "JAVA|11"

Output:

Web App successfully created.

Configure Deployment Task in Azure Pipelines

Modify azure-pipelines.yml:

```
trigger:
- main
pool:
  vmImage: 'ubuntu-latest'
steps:
- task: Maven@3
  inputs:
    mavenPomFile: 'pom.xml'
    goals: 'clean package'
- task: AzureWebApp@1
  inputs:
    azureSubscription: 'MyServiceConnection'
    appName: 'MyAppService'
```

package: '$(System.DefaultWorkingDirectory)/target/*.jar'

Output:

Deployment started... Application successfully deployed to Azure.

Step 5: Configuration Management with Ansible

Install Ansible

sudo apt update && sudo apt install ansible -y

Output:

Ansible installed successfully.

Create an Ansible Playbook (deploy.yml)

```
- name: Deploy Application
  hosts: localhost
  become: yes
  tasks:
    - name: Copy JAR to server
      copy:
        src: /var/lib/jenkins/workspace/Maven-CI/target/myapp.jar
        dest: /opt/myapp.jar
    - name: Start Application
      command: nohup java -jar /opt/myapp.jar &
```

Output:

Ansible Playbook executed, application deployed successfully.

Step 6: Best Practices Discussion

- CI/CD Best Practices:
  - ✔ Automate testing and security checks
  - ✔ Use infrastructure as code (IaC)
  - ✔ Implement versioning and rollback strategies
- Security Best Practices:
  - ✔ Use Azure Key Vault for storing credentials
  - ✔ Apply least privilege access to DevOps tools

Viva- Questions:

1. What are the key differences between Maven and Gradle?
2. What is the purpose of a POM file in Maven?
3. How does dependency management work in Maven and Gradle?
4. What is the Gradle Wrapper, and why is it useful?
5. What are Maven phases? Name a few.
6. What is the difference between `build.gradle` and `build.gradle.kts`?

7. How do you define a custom task in Gradle?
8. How do you migrate a Maven project to Gradle?
9. What is Jenkins, and why is it used in DevOps?
10. How do you set up a CI pipeline in Jenkins?
11. What is a Jenkinsfile, and why is it important?
12. How do you integrate Maven/Gradle with Jenkins?
13. What is Ansible, and how does it help in automation?
14. What are Ansible Playbooks, and how do they work?
15. What is idempotency in Ansible, and why is it important?
16. What is Azure DevOps, and what are its core services?
17. How do you create a build pipeline in Azure DevOps?
18. What is the purpose of Azure Key Vault in DevOps?
19. How does Continuous Deployment (CD) work in Azure Pipelines?
20. How do you monitor and troubleshoot failures in a CI/CD pipeline?